

큐기출

오답을 분석하고, 취약점을 파악하고,
AI가 맞춤 문제를 생성하는 자격증 CBT 학습 플랫폼.

발표 순서

01 왜 시작했나 (Why)

수험생이 지금 겪는 3가지 고통

03 실제 화면 데모

랜딩 · 대시보드 · CBT 시험 · 결과 분석

05 프로젝트 일정 & R&R

4주 WBS와 팀원별 담당 영역

02 무엇을 만들었나 (What)

시험 응시 · AI 분석 · AI 문제 생성 · 오답 노트

04 어떻게 만들었나 (How)

아키텍처 · 기술 스택 · ERD · AI 파이프라인

06 회고 & 다음 단계

배운 점과 v2.0 고도화 방향

I.

왜 만들었나

자격증 시험을 준비하는 수험생은 여전히 단순 반복 풀이에 머문다.
틀린 이유도, 약한 단원도 스스로 정리해야 하는 것이 현실이다.

수험생이 지금 겪는 3가지 고통

단순 반복 학습이 놓치는 세 가지 지점입니다.

01

파편화된 기출문제

기출 자료가 PDF·게시글·영상에 흩어져 있어, 풀 문제를 한곳에 모으기가 번거롭다.

자료가 흩어짐

02

내 약점을 모른다

틀린 문제를 따로 기록하지 않으면 같은 단원에서 또 틀린다. 오답 정리는 100% 수작업.

반복 실패 패턴

03

실전 감각 부재

실제 시험은 컴퓨터(CBT)로 진행되는데 종이 프린트로만 연습 → 시험 당일 UI 적응에 시간 소모.

CBT 환경 미숙

II.

무엇을 만들었나

문제를 푸는 것에서 끝나지 않는다.

왜 틀렸는지 — 어디가 약한지 — 무엇을 더 공부할지까지.

합격 루틴을 완성하는 3단계 학습 사이클

STEP 1 · DIAGNOSE

왜 틀렸지?

실전 CBT 모드로 풀이하면 오답이 자동 저장된다. 채점에 그치지 않고 풀이 시간
까지 기록한다.



STEP 2 · ANALYZE

어디가 약하지?

GPT-4o가 오답 패턴을 분석해 취약 단원과 오류 유형을 자동 도출한다.



STEP 3 · REINFORCE

뭘 더 풀지?

취약 단원의 유사 문제를 AI가 자동 생성. 같은 실수를 반복하지 않도록 집중 학
습한다.

이중 모드 — 실전과 학습 사이

같은 문제 DB 위에서 두 가지 풀이 경험을 제공한다.

⚡ 시험 모드

실전 감각을 훈련한다

실제 CBT와 동일한 화면에서 서버 타이머가 동작.
제출 전에는 정답도, 해설도 볼 수 없다.

타이머	실제 시험 시간 적용 (150분)
정답 확인	제출 후 일괄 공개
해설	제출 후 공개
나중에 풀기	플래그로 표시 후 일괄 검토

📖 연습 모드

개념을 이해하고 반복한다

한 문항씩 즉시 채점·해설.
오답이 나오면 그 자리에서 다시 풀 수 있다.

타이머	없음 — 학습 페이스 자유
정답 확인	문항별 즉시 공개
해설	오답 시 즉시 팝업
오답 재풀이	해설 확인 후 같은 문항 재시도

전체 기능 한눈에



자격증 시험 응시

자격증별 기출문제를 타이머와 함께 풀어볼 수 있는 CBT 풀이 엔진.

/api/sessions/start



성적 분석 차트

과목별·단원별 정답률을 레이다·라인 차트로 시각화 (Recharts).

/api/stats



AI 약점 분석

오답 데이터를 GPT-4o에 전달 → 취약 단원·오류 유형 자동 도출.

/api/ai/analysis



AI 문제 생성

취약 단원과 같은 난이도·유형의 신규 문제를 자동 생성해 반복 학습 지원.

/api/ai/generate-questions



오답 노트

틀린 문제 자동 저장 + 내 메모 추가. 오답만 모아 재시험 가능.

/api/notes



학습 대시보드

누적 풀이·연속 학습·평균 점수 등 학습 KPI를 한 화면에 집약.

/api/stats/summary

III.

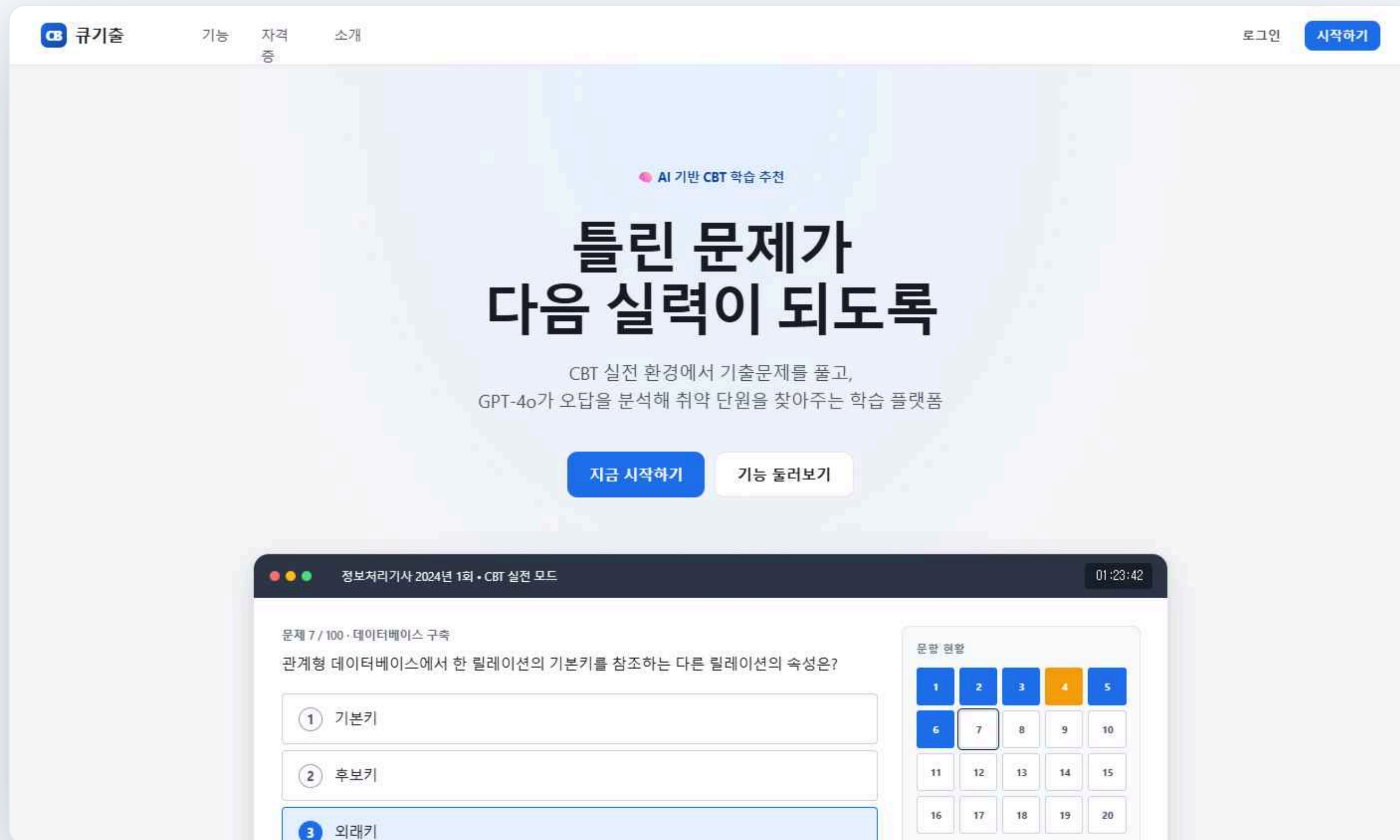
실제 화면

완성된 서비스의 핵심 6개 화면을 차례로 살펴봅니다.

합격까지 가장 짧은 길.

서비스의 핵심 가치를 한눈에 전달하는 랜딩. 실제 CBT 풀이 화면 미리보기를 그대로 노출한다.

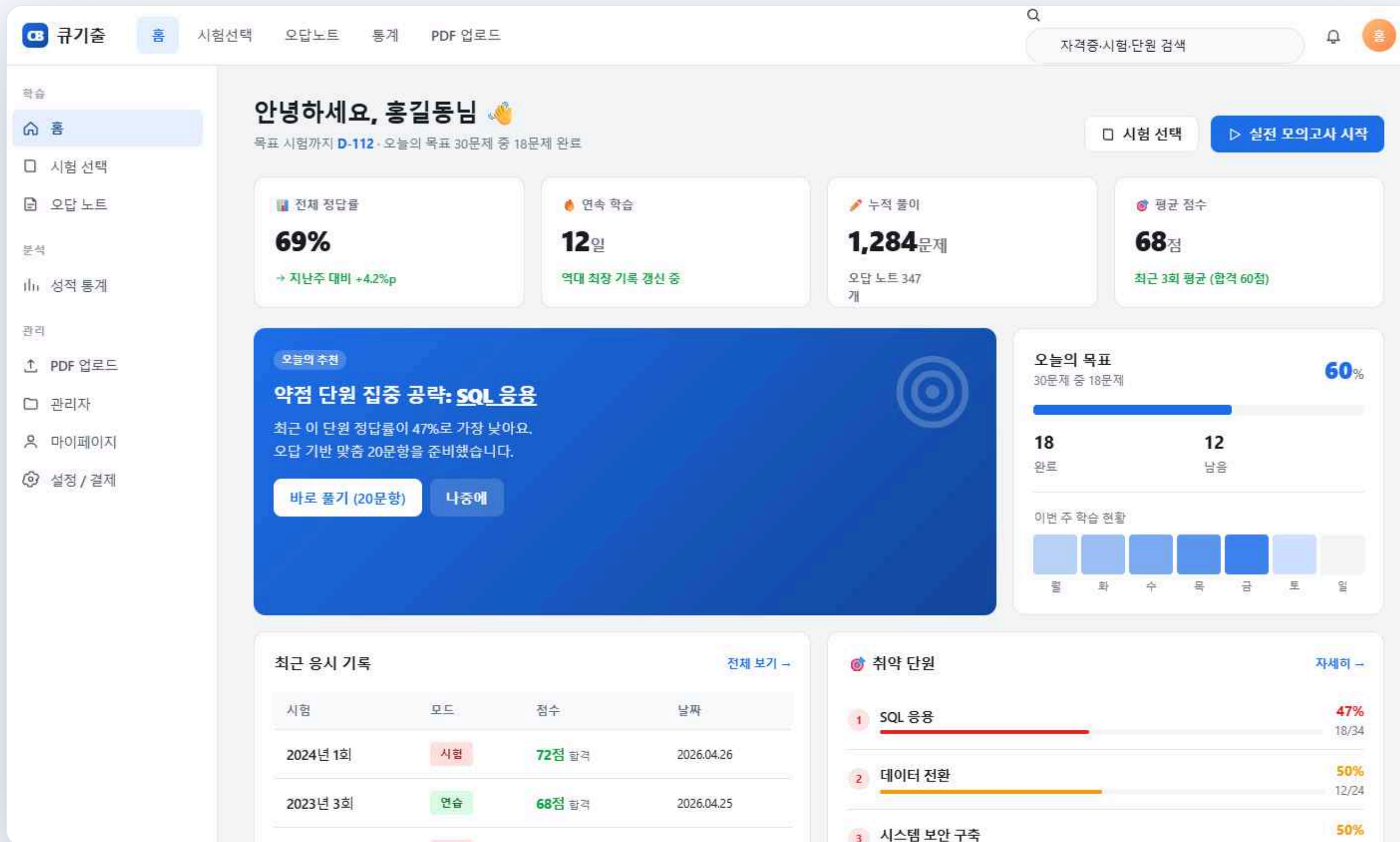
- Hero · 핵심 가치 제안 + 시작 CTA
- 실제 CBT 풀이 화면 임베드 프리뷰
- 주요 기능 · 지원 자격증 소개



학습의 HQ가 된다.

로그인 직후 마주하는 학습 본부. KPI 4종, 오늘의 추천 학습, 취약 단위, 최근 응시 기록을 한 화면에 정리했다.

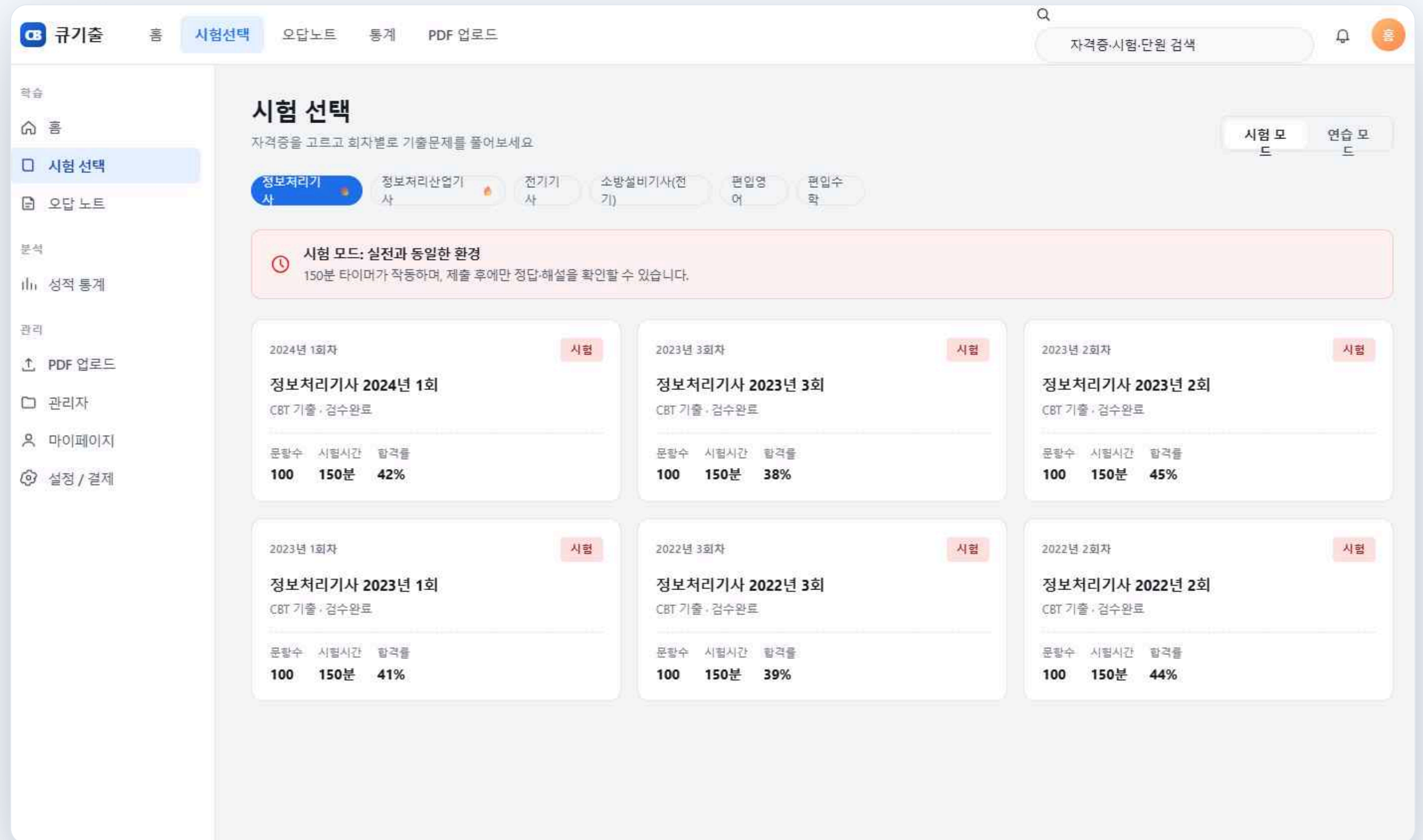
- 전체 정답률 · 연속 학습 · 누적 풀이 · 평균 점수
- "오늘의 추천" 카드 — AI가 풀 문제까지 큐잉
- 주간 학습 현황 히트맵 + 일일 목표 진척도



자격증 → 회차 2-step 선택.

자격증을 칩으로 고르고, 회차 카드에서 시험·연습 모드를 선택. 모드별로 안내 컬러와 카피가 달라져 잘못된 진입을 방지한다.

- 6개 자격증 카테고리 · 회차별 합격률 표시
- 시험/연습 모드 세그먼트 토글
- 모드별 안내 배너 — 타이머·해설 정책 안내



실전 CBT, 그대로 재현.

실제 CBT 시험장과 동일한 UI로 구성. 다크 헤더의 타이머, 우측 문항 그리드, 제출 버튼까지 실전과 같은 배치.

- 서버 기반 타이머 — 새로고침해도 시간 유지
- 20문항 그리드 · 답안완료/체크/미응시 색상 코드
- "나중에 풀기" 플래그 + 일괄 검토 모달

시험 모드정보처리기사 2024년 1회응시번호 2026042600128 · 홍길동

01:29:59

나가기

문제 1번

소프트웨어 설계요구사항 확인

나중에

다음 중 애자일(Agile) 개발 방법론의 특징으로 가장 거리가 먼 것은?

1

변화하는 요구사항에 빠르게 대응한다

2

짧은 주기(Sprint)로 반복 개발을 진행한다

3

고객과의 지속적인 협력을 중요시한다

4

초기에 모든 요구사항을 확정하고 변경을 최소화한다

문항 바로가기

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

답안완료

체크(나중에)

미응시

답안 제출

이전 문제

다음 문제

풀이 진행률0%

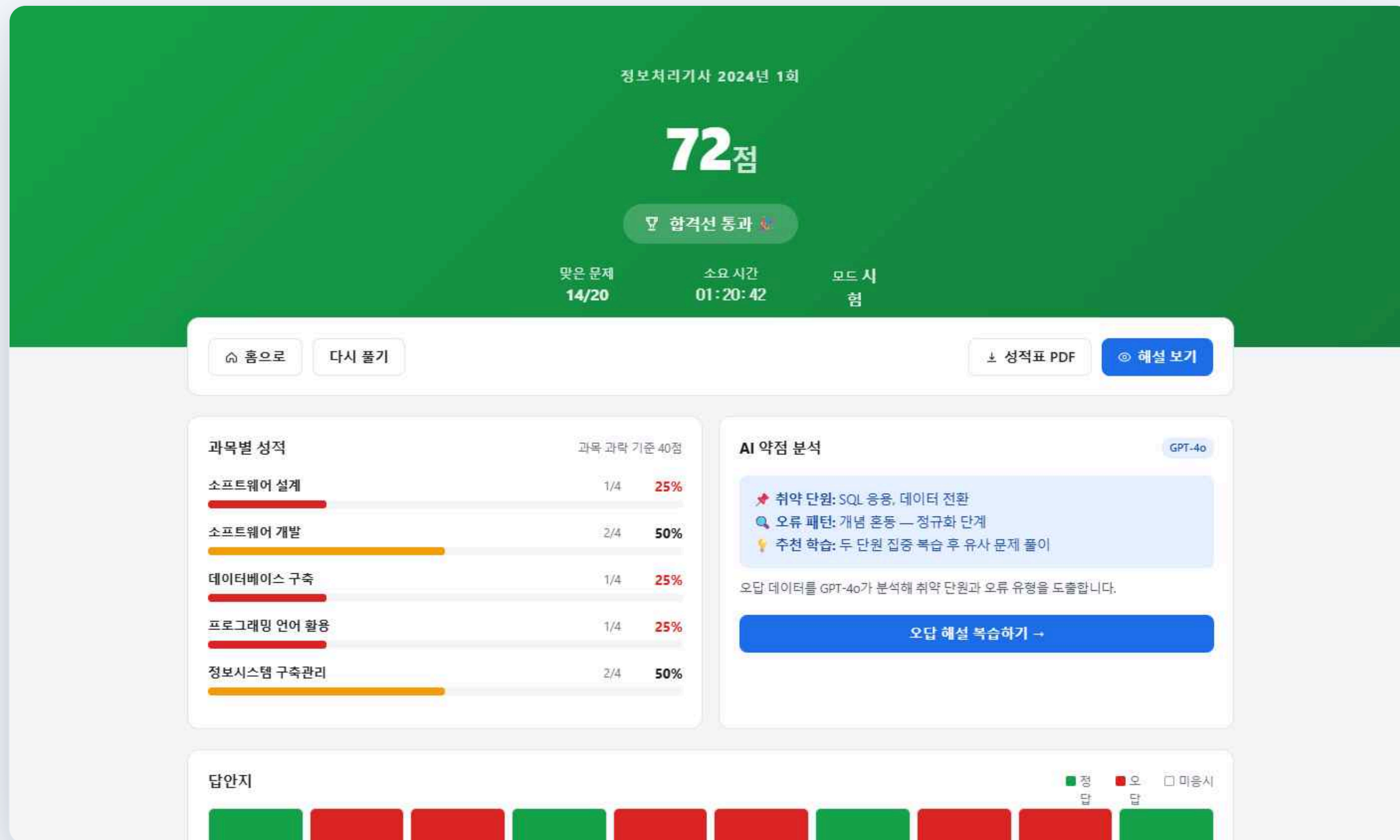
체크

답안 제출

점수보다 "왜"를 본다.

합격 여부만 보여주는 결과 화면이 아니다. GPT-4o가 즉시 취약 단원과 오류 패턴을 분석해 다음 학습 액션을 제안한다.

- 총점 · 과목 과락 · 합격선 통과 여부 한 화면 요약
- AI 분석 카드: 취약 단원 + 오류 패턴 + 추천 학습
- 답안지 시각화 + 오답 해설 복습 진입점



오답 정리, 이제 자동으로.

틀린 문제는 자동으로 노트에 쌓인다. 사용자는 메모만 남기면 된다. "오답만 재시험" 한 번에 진입.

- 마스터-디테일 레이아웃 · 좌측 리스트 + 우측 상세
- 정답·내 답·해설·내 메모 4단 구성
- "오답만 재시험" · "다시 풀기" 원클릭 액션

CB 큐기출

홈

시험선택

오답노트

통계

PDF 업로드

자격증·시험·단원 검색

🔔

👤

학습

홈

시험 선택

오답 노트

분석

성적 통계

관리

PDF 업로드

관리자

마이페이지

설정 / 결제

오답 노트

틀린 문제 자동 저장 · 재풀이로 완벽 숙지

🔍 필터

▶ 오답만 재시험

전체 10개

최신순

소프트웨어 설계

2회 틀림

다음 중 애자일(Agile) 개발 방법론의 특징으로 가장 거리가 먼 것은?

요구사항 확인 · 04.20

소프트웨어 설계

2회 틀림

UI 설계 원칙 중 "사용자가 서비스를 쉽게 이해하고 사용할 수 있어야 한다"는 원칙은?

화면 설계 · 04.21

소프트웨어 설계

2회 틀림

디자인 패턴 중 객체의 생성을 서브클래스에서 결정하도록 하는 생성 패턴은?

애플리케이션 설계 · 04.22

소프트웨어 개발

2회 틀림

소프트웨어 통합 테스트 방법 중 상위 모듈부터 하위 모듈 방향으로 테스트를 진행하는 방식은?

통합 구현 · 04.23

소프트웨어 개발

2회 틀림

소프트웨어 형상관리(Configuration Management)의 활동으로 옳지 않은 것은?

데이터베이스 구축

SQL 응용

2회 오답

☆

🗑

Q. 다음 중 정규화 과정에서 제2정규형의 조건은?

① 도메인이 원자값만을 갖는다

2

부분 함수 종속성을 제거한다 ✓ 정답

③ 이행적 함수 종속성을 제거한다

④ 모든 결정자가 후보키이다

💡 해설

제2정규형(2NF)은 제1정규형을 만족하고 부분 함수 종속을 제거한 상태입니다. 3NF는 이행적 종속, BCNF는 결정자/후보키 조건.

📝 내 메모

정규화는 매번 헛갈린다. 2NF는 '부분 함수 종속 제거', 3NF는 '이행적 종속 제거'로 외우자.

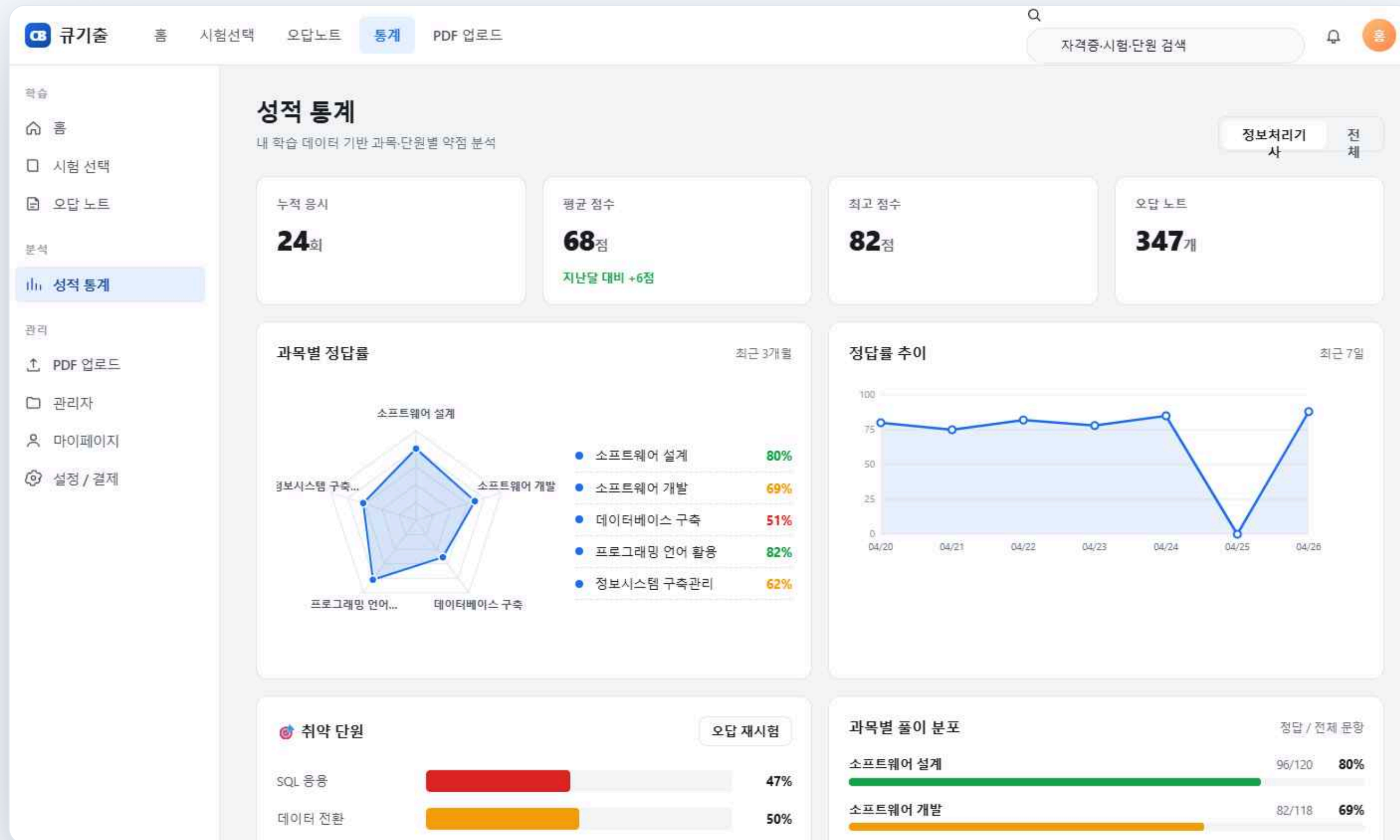
이전 오답

다시 풀기 →

레이더 차트로 강·약점 한눈에.

응시 횟수가 쌓일수록 데이터는 더 정확해진다.
과목별 레이더, 정답률 추이, 단위별 분포까지
Recharts로 시각화한다.

- 과목별 정답률 레이더 차트
- 최근 7일 정답률 추이 라인 차트
- 취약 단위 + 과목별 풀이 분포



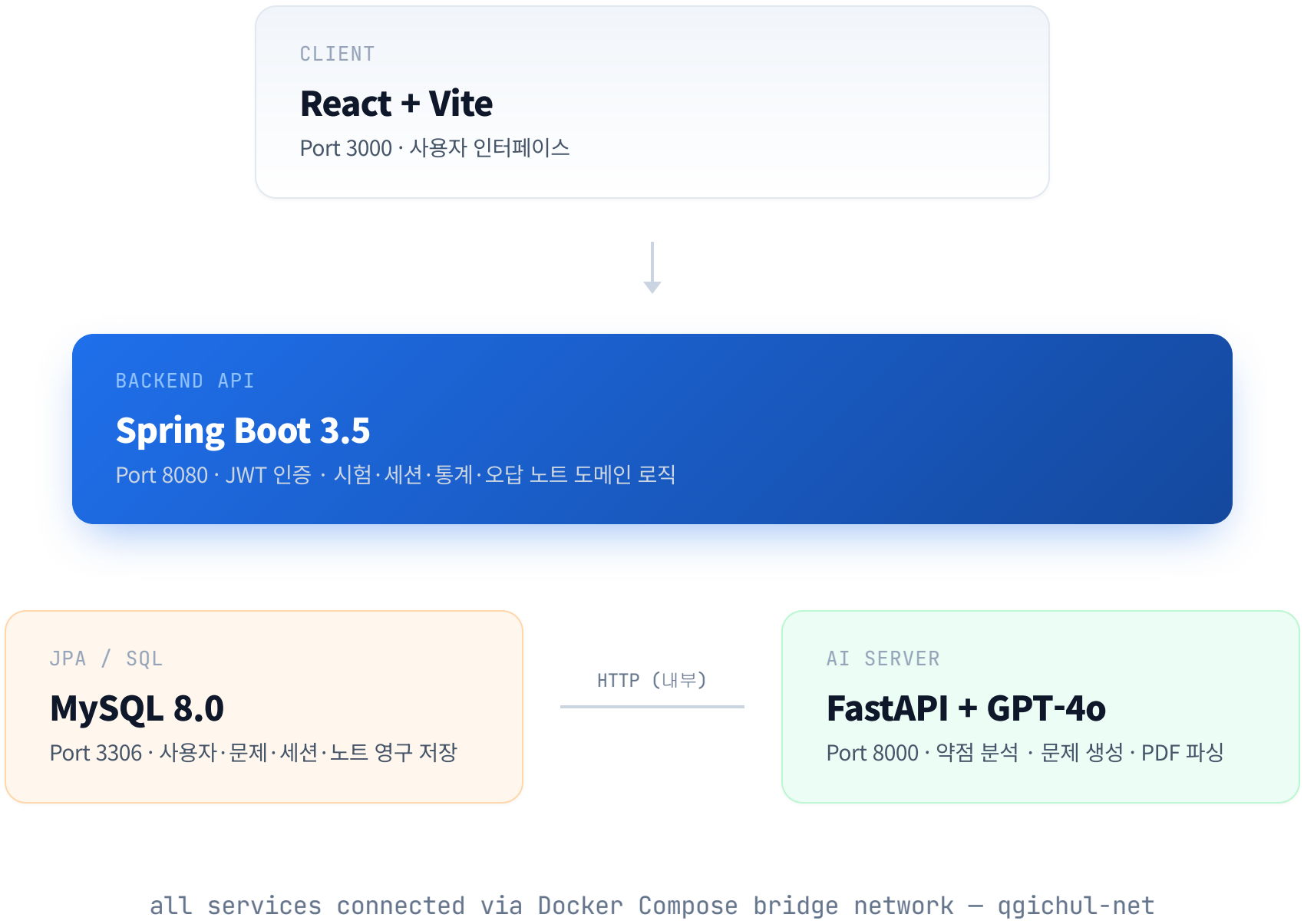
IV.

어떻게 만들었나

3개의 서비스, 1개의 네트워크.

React · Spring Boot · FastAPI를 Docker Compose 위에 묶었다.

시스템 아키텍처



기술 스택

FRONTEND			BACKEND			AI SERVER			DEVOPS		
React			Java			Python			Docker Compose		
Vite			Spring Boot			FastAPI			Gradle		
React Router			Spring Security · JWT			OpenAI			GitHub		
Axios			Spring Data JPA			pdfplumber			Bridge Network		
Recharts			MySQL			Uvicorn			.env Config		

데이터 모델 (개념 ERD)

사용자 풀이 행위(USER_ANSWERS)가 모든 통계의 원천이 되도록 정규화했다.

USERS 유저		
PK	user_id	UUID
	email	VARCHAR
	password_hash	VARCHAR
	plan_type	ENUM
	created_at	TS

EXAMS 시험		
PK	exam_id	UUID
	name	VARCHAR
	category	VARCHAR
	year	INT
	round	INT

QUESTIONS 문항		
PK	question_id	UUID
FK	exam_id	→ EXAMS
FK	subject_id	→ SUBJECTS
	content	TEXT
	answer	INT
	unit_tag	VARCHAR

SESSIONS 응시세션		
PK	session_id	UUID
FK	user_id	→ USERS
FK	exam_id	→ EXAMS
	mode	ENUM
	score	INT

USER_ANSWERS 통계원천		
PK	answer_id	UUID
FK	session_id	→ SESSIONS
FK	question_id	→ QUESTIONS
	selected	INT
	is_correct	BOOL
	time_spent	INT (sec)

WRONG_NOTE 오답노트		
PK	note_id	UUID
FK	user_id	→ USERS
FK	question_id	→ QUESTIONS
	memo	TEXT
	created_at	TS

AI 분석·문제 생성 파이프라인

시험 제출 → 오답 데이터 패키징 → GPT-4o 분석 → 유사 문제 생성까지 한 흐름으로 묶었다.

STEP 01

오답 추출

USER_ANSWERS에서 is_correct=false인 문항을 단위·과목·풀이 시간과 함께 패키징.

Spring Boot

STEP 02

프롬프트 빌드

오답 JSON을 prompts/ 템플릿에 주입해 분석 지시 프롬프트를 구성.

FastAPI · prompts/

STEP 03

GPT-4o 분석

취약 단원과 오류 유형, 학습 우선순위를 구조화 JSON으로 반환.

OpenAI · GPT-4o

STEP 04

유사 문제 생성

취약 단원과 같은 난이도·형식의 신규 문제를 GPT가 생성해 클라이언트에 즉시 노출.

/api/ai/generate-questions

// 예시: GPT-4o 응답 구조

```
{ "weak_chapters": ["직류기", "유도전동기"], "error_patterns": ["개념 혼동: 역기전력 공식"], "recommendations": ["직류기 단위 집중 복습 후 유사 문제 풀이"] }
```


Next.js SSR → React CSR 전환

초기 SSR 구조에서 부딪힌 문제를 진단하고, CSR로 전환해 해결한 과정.

PROBLEM · 문제

SSR 경계에서 반복되는 오류

타이머·문항 네비게이션·실시간 답안 상태가 얇힌 CBT 화면에서 서버/클라이언트 컴포넌트 경계를 나누다 **useEffect·useState 오류**가 반복됐다. JWT 키키 처리도 서버·클라이언트 방식 차이로 인증 오류가 발생.

CAUSE · 원인

인터랙션 복잡도가 SSR 이점을 압도

CBT 핵심 화면은 SSR의 초기 로딩 이점보다 **클라이언트 인터랙션 복잡도**가 훨씬 컸다. SSR을 유지하며 그 복잡도를 제어하는 비용이 CSR 전환 비용보다 크다고 판단했다.

SOLUTION · 해결

React + Vite CSR로 전환

번들러를 **Vite**로 교체해 개발 서버 HMR 속도가 개선됐고, 시험 응시 화면의 상태 관리 구조가 단순해져 유지보수성이 높아졌다.

프로젝트 일정

4주에 걸쳐 기획부터 통합까지 마무리. 매주 마일스톤이 명확했다.

04/13 - 04/17	Week 1 · 기획 & 설계 요구사항 분석, 기술스택 선정, ERD 확정, UI 와이어프레임. 팀 R&R 결정.
04/27 - 05/01	Week 2 · 백엔드 & 프론트엔드 골격 Spring Boot 인증·시험·세션 API 구축, MySQL 스키마 적용, 프론트엔드 화면 골격 완성.
05/04 - 05/08	Week 3 · 백엔드 마무리 & AI 개발 통계·오답 노트 API 완성. FastAPI 약점 분석 / 문제 생성 라우터, GPT-4o 프롬프트 튜닝.
05/11	Week 4 · 통합 & 발표 준비 Docker Compose 통합, 전체 흐름 QA, 데모 시나리오 정리, 발표 자료 완성.

팀원 및 역할

3인 1팀, 각자 한 도메인을 책임지고 매주 통합 점검.



박성훈

FRONTEND & ARCHITECTURE

- 시스템 아키텍처 설계
- 프론트엔드(React) 개발
- Spring Boot 백엔드 개발



강민구

AI ENGINEER

- FastAPI AI 서버 개발
- GPT-4o 프롬프트 엔지니어링
- 약점 분석 · 유사 문제 생성 로직
- pdfplumber 기반 PDF 파싱



유찬양

BACKEND ENGINEER

- Spring Boot 백엔드 개발
- DB 설계 · MySQL 스키마
- JPA 매핑 · 데이터 접근 계층

감사합니다

"왜 틀렸는지 → 어디가 약한지 → 무엇을 더 공부할지" —
큐기출이 합격까지의 거리를 가장 짧게 만들겠습니다.

박성훈 · Frontend & Architecture

강민구 · AI Engineer

유찬양 · Backend Engineer

SMU 엔지니어 9기 · 팀 미니 프로젝트 · 2026.05